Bio Bloom Test Case Register and Coverage Report

Part A: Test Case Register. Part B: Coverage Report. Measured on 25 October 2026.

Item	Detail
Project	Bio Bloom: Artisanal Food E-Commerce Platform
Course	SENG 34213
Department	Department of Software Engineering, Faculty of Science, University of Kelaniya
Document	Test Case Register and Coverage Report
Version	1.0
Date	25 October 2026
Status	Final for submission
Team	Ovindu Pathirana (SE/2021/059), Prageeth Ekanayake (SE/2021/049): backend, database, cron, security, DevOps. Chamindu Dilshan (SE/2021/058), Sanduni Ayeshika (SE/2021/039): frontend. I.G.R. Ravindu Laksara (SE/2021/028): mobile
Reference code	Monorepo `biobloom`: API `apps/api` (unit specs in `apps/api/src/**/*.spec.ts`, Supertest suites in `apps/api/test`), web apps `apps/storefront`, `apps/admin`, `apps/seller`, shared UI `packages/ui`, mobile `apps/delivery`
Related	SDS v01 §8.2 (pipeline stages 3 and 4), backlog tickets DDP-#33 (unit coverage gate) and DDP-#34 (integration suite in CI), `apps/api/jest.config.json`, `apps/api/test/jest-e2e.json`, `.github/workflows/ci.yml`

Version history

Version	Date	Author	Summary
1.0	25 October 2026	Project team	First register built from the real test files, with measured results and coverage data

Table of contents

1. Executive summary
2. Scope, test environment and status rules
3. Test levels and tools
4. Part A: Test Case Register
5. Analysis of the failing integration tests
6. Part B: Coverage Report
7. Supertest integration suites
8. CI status
9. Plan to reach the coverage gate
10. Acceptance check for DDP-#33 and DDP-#34
11. Evidence index

1. Executive summary

We ran every automated test that could run on a single machine on 25 October 2026. The results are below. Nothing in this report is estimated: each number comes from a test run or from the repository.

Area	Measured result	Target	Verdict
API unit tests (Jest)	24 suites, 231 tests, 231 pass	All pass	Met
API unit coverage (statements)	31.24% (1198 of 3834)	65% gate (`jest.config.json`), 80% target	**Not met**
API unit coverage (branches)	23.38% (348 of 1488)	50% gate, 80% target	**Not met**

Area	Measured result	Target	Verdict
API unit coverage (functions)	21.82% (163 of 747)	65% gate, 80% target	**Not met**
API unit coverage (lines)	31.54% (1085 of 3440)	65% gate, 80% target	**Not met**
Web unit tests (Vitest)	40 tests in 11 files (storefront 11, admin 7, seller 2, `packages/ui` 20), all pass	All pass	Met
API integration tests (Supertest)	11 suites, 36 tests: 31 pass, 5 fail	All pass	**Not met**
Browser end-to-end tests (Playwright)	Not run in this measurement	Critical journeys pass	Not measured
Mobile tests (`flutter test`)	Not run in this measurement	All pass	Not measured
Monorepo CI (`ovindu17/biobloom`)	Fails in the first job (actionlint SC2034 in `.github/workflows/deploy.yml` line 90); test jobs skipped	Green pipeline	**Not met**
Team backend repository (separate)	develop `55e9ebb`: 338 tests pass, 91.27% statements under its own 80% gate	80%	Met (team repo only)

Test Case Register in Part A: **94 test cases**. **73 PASS, 5 FAIL, 16 NOT RUN.**

Key points:

- The unit tests that exist all pass, but they cover less than half of what the gate needs. Ten of the 29 API modules have **0%** line coverage. The largest of these are `products` (313 lines), `bundles` (156), `cart` (140) and `refunds` (105).
- Five integration tests fail. Our analysis (Section 5) finds that four are **test defects**: a test sets the payment mode too late, a test assumes a rider has no seeded assignments, a test expects an older refund response, and a test uses a non-UUID id. None of them points to a confirmed product bug. They do hide real behaviour, though. For example, no passing test proves that a Stripe signature is checked in live mode.
- The CI pipeline has not run the unit, audit or integration jobs on recent commits, because the first job fails on one shell lint warning.

---

2. Scope, test environment and status rules

2.1 Scope

- API unit tests: `apps/api/src/**/*.spec.ts` (24 files).
- API integration tests: `apps/api/test/*.e2e-spec.ts` (11 files).
- Web unit tests: Vitest files in `apps/storefront`, `apps/admin`, `apps/seller` and `packages/ui` (11 files).
- Browser tests: Playwright files in `apps/*/e2e` (6 files). Listed, not run.
- Mobile tests: `apps/delivery/test` (7 files). Listed, not run.

2.2 Measurement environment

Item	Value
Date	25 October 2026
Machine	Apple M1, 8 GB RAM
Runtime	Node 26 (CI uses Node 20 from `.nvmrc`)
Database	PostgreSQL 16, local instance (CI uses the `postgres:15-alpine` service container)
Cache	Redis 7, local instance (CI uses `redis:7-alpine`)
Mail	MailHog v1.0.1 (CI uses `mailhog/mailhog:latest`)
Integration setup	Same environment variables as the `integration` job in `.github/workflows/ci.yml` (including `PAYMENTS_MODE=mock`, `TZ=UTC`), then `prisma migrate deploy` and the seed
Unit commands	`pnpm --filter @biobloom/api test:cov`, then `test` in each web workspace

Item	Value
Integration	`pnpm --filter @biobloom/api test:e2e`. The notifications suite was run as its own invocation (1 test). The other 10 suites ran together (35 tests).
Re-run check	The three failing suites that do not need MailHog (webhook-signature, delivery, refund) also fail when run alone on a freshly migrated and seeded database. The failures do not depend on run order.

2.3 Status rules

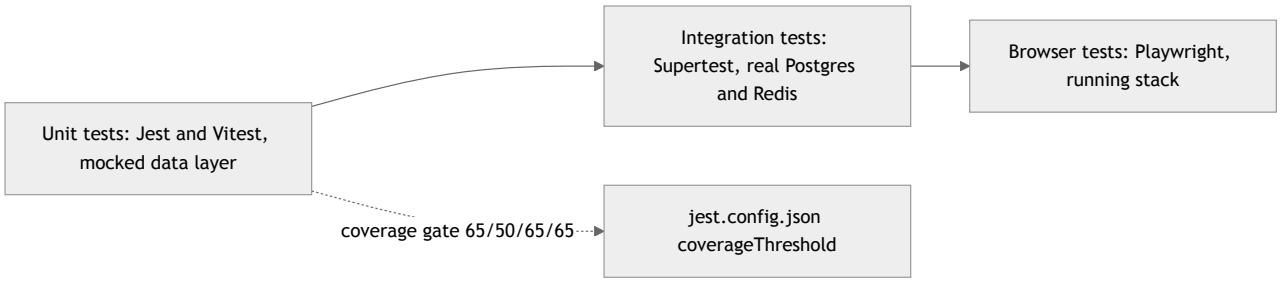
Status	Meaning
PASS	The test ran on 25 October 2026 and passed.
FAIL	The test ran on 25 October 2026 and failed. The failure message is quoted in "Actual Result".
NOT RUN	The test exists but was not run in this measurement, or there is no automated test yet ("No automated test yet").

A case is never marked PASS because it "should" pass or passed earlier. The register lists a sample of the 231 API unit tests and the 40 Vitest tests, chosen to cover each module and every boundary case the backlog asks for. Every integration test is listed.

---

3. Test levels and tools

Level	Tool	Location	Runs against	In CI (`ci.yml`)
Unit (API)	Jest 29 with `ts-jest`	`apps/api/src/**/*.spec.ts`	Plain objects in place of `PrismaService` and Redis	Job `unit`, with coverage gate
Unit (web)	Vitest 2 with Testing Library	`apps/*/**/*.test.ts(x)`, `packages/ui/src/components/__tests__`	jsdom	Job `unit`
Unit and widget (mobile)	`flutter test`	`apps/delivery/test`	Mocked HTTP client	Job `flutter`
Integration (API)	Jest with Supertest	`apps/api/test/*.e2e-spec.ts`	The full Nest app, real PostgreSQL, Redis and MailHog	Job `integration`, service containers
End to end (browser)	Playwright 1.49 with axe checks	`apps/storefront/e2e`, `apps/admin/e2e`, `apps/seller/e2e`	A running stack (Docker Compose)	Not in `ci.yml`



---

4. Part A: Test Case Register

4.1 How to read the register

- **Test Case ID**: `TC-U` unit, `TC-I` integration, `TC-E` browser end to end, `TC-G` gap (no automated test yet).
- **Test Scenario** ends with the source: the spec file and the exact test name in quotes.
- **Status** follows the rules in Section 2.3.

4.2 API unit tests (Jest)

All 231 API unit tests passed in the 25 October 2026 run, so every case below is PASS.

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-01	Unit	auth	Login gives the same error for an unknown email and a wrong password. Source: `apps/api/src/auth/auth.service.spec.ts`, "rejects unknown email with the same exception as a wrong password"	Prisma and Redis mocked; no account with the email	1) Call `login` with an unknown email. 2) Call `login` with a known email and a wrong password.	Both calls throw the same `UnauthorizedException`; the error does not reveal which email exists	Same exception in both calls	PASS
TC-U-02	Unit	auth	Registration hashes the password with bcrypt cost 12 and lowercases the email. Source: `auth.service.spec.ts`, "hashes with bcrypt cost 12, lowercases email, and persists"	No customer, admin, seller or rider with the email	Call `registerCustomer` with a mixed-case email	Saved email is lowercase; hash uses cost 12	As expected	PASS
TC-U-03	Unit	auth	The roles guard refuses a role that the route does not allow. Source: `apps/api/src/auth/guards/roles.guard.spec.ts`, "rejects another role with 403 naming the role"	Route marked with `@Roles`; caller has another role	Run `canActivate`	`ForbiddenException` (403) naming the role	As expected	PASS
TC-U-04	Unit	auth	A revoked token is refused before the account is loaded. Source: `apps/api/src/auth/strategies/jwt.strategy.spec.ts`, "rejects a revoked token before loading the account"	Token JTI is on the Redis deny list	Call `validate` with the token payload	401; the database is not queried	As expected	PASS
TC-U-05	Unit	common	CSRF check refuses a POST whose header does not match the cookie. Source: `apps/api/src/common/middleware/csrf.middleware.spec.ts`, "rejects a POST whose header does not match the cookie"	`csrf-token` cookie present	Send POST with a different `x-csrf-token` header	403	As expected	PASS
TC-U-06	Unit	inventory	Zero stock: available stock is 0 when no valid batch exists. Source: `apps/api/src/inventory/inventory.service.spec.ts`, "returns 0 when no valid batches exist"	Batch aggregate returns no sum	Call `availableStock`	Returns 0	Returned 0	PASS
TC-U-07	Unit	inventory	FIFO takes stock across batches when the earliest one is short. Source: `inventory.service.spec.ts`, "spans across batches when earliest is insufficient (SDS §5.3 atomicity)"	Two batches ordered by expiry date	Call `decrementForVariant` with more than the first batch holds	First batch drained and DEPLETED; the rest taken from the next batch	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-08	Unit	inventory	Zero stock: no update is made when there is no valid batch. Source: `inventory.service.spec.ts`, "throws OutOfStockError with NO updates when zero valid batches found"	No valid batches	Call `decrementForVariant`	`OutOfStockError`; no batch update	As expected	PASS
TC-U-09	Unit	inventory	Quantity 0 or less is refused. Source: `inventory.service.spec.ts`, "rejects non-positive qty before touching the DB"	None	Call `decrementForVariant` with a non-positive quantity	Error before any database call	As expected	PASS
TC-U-10	Unit	shipping	Weight bracket edge: 500 g. Source: `apps/api/src/shipping/shipping.service.spec.ts`, "weight %d g → %s LKR (%s)" row 500	Brackets as in the seed: 0–500 g 350.00, 501–1000 g 500.00, 1001–2000 g 750.00, 2001–5000 g 1100.00, 5001 g and up 1800.00	Call `calculate` with one 500 g item	Fee 350.00, label "0–500 g"	350.00, "0–500 g"	PASS
TC-U-11	Unit	shipping	Weight bracket edge: 501 g. Source: `shipping.service.spec.ts`, row 501	Same brackets	Call `calculate` with one 501 g item	Fee 500.00, label "501 g – 1 kg"	500.00, "501 g – 1 kg"	PASS
TC-U-12	Unit	shipping	Weight bracket edge: 1001 g. Source: `shipping.service.spec.ts`, row 1001	Same brackets	Call `calculate` with one 1001 g item	Fee 750.00, label "1–2 kg"	750.00, "1–2 kg"	PASS
TC-U-13	Unit	shipping	Lower edge: 0 g. Source: `shipping.service.spec.ts`, row 0	Same brackets	Call `calculate` with one 0 g item	Fee 350.00	350.00	PASS
TC-U-14	Unit	shipping	No bracket covers the weight. Source: `shipping.service.spec.ts`, "throws NotFound when no bracket covers the weight"	Bracket table returns no rows	Call `calculate` with 100 g	`NotFoundException`	As expected	PASS
TC-U-15	Unit	shipping	Admin bracket edit with a gap is refused. Source: `apps/api/src/shipping/shipping-rates-admin.service.spec.ts`, "rejects a gap between brackets"	None	Call `bracketProblem` with 0–500 g and 600–1000 g	Message "501–599 g are not covered"	As expected	PASS
TC-U-16	Unit	batches	Expired batch: the daily sweep marks it EXPIRED. Source: `apps/api/src/batches/expiry-cron.service.spec.ts`, "marks already-expired batches EXPIRED and emits batch.expired"	One batch expired 2 days ago	Call `sweep`	`expiredCount` 1; status EXPIRED; `batch.expired` event sent	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-17	Unit	batches	A batch 3 days from expiry is marked EXPIRING_SOON. Source: `expiry-cron.service.spec.ts`, "marks within-7-day batches EXPIRING_SOON and emits with daysUntilExpiry"	One batch expiring in 3 days	Call `sweep`	`expiringSoonCount` 1; event with `daysUntilExpiry` 3	As expected	PASS
TC-U-18	Unit	payments	Payment failure restock: a failed payment cancels the order and returns stock. Source: `apps/api/src/payments/payments.service.spec.ts`, "on failure cancels the order and restocks through InventoryService"	PENDING payment for an order in PENDING_PAYMENT	Call `applyPaymentEvent` with outcome failed, reason `card_declined`	`restockOrderItems` called; order moved to CANCELLED with the reason; cancellation notice sent	As expected	PASS
TC-U-19	Unit	payments	A lost race on the PENDING claim is a duplicate, so stock is not returned twice. Source: `payments.service.spec.ts`, "treats a lost race on the PENDING claim as a duplicate (no double restock)"	Conditional update claims 0 rows	Call `applyPaymentEvent`	Result `duplicate`; no restock	As expected	PASS
TC-U-20	Unit	payments	A success with the wrong amount is refused. Source: `payments.service.spec.ts`, "rejects a success whose amount does not match the payment"	PENDING payment of a fixed amount	Apply a success event for 1.00	`BadRequestException`	As expected	PASS
TC-U-21	Unit	payments	A payment that arrives after the order was cancelled is refunded. Source: `payments.service.spec.ts`, "refunds a payment that arrives after the order was cancelled instead of marking it PAID"	Order already CANCELLED	Apply a success event	No order transition; payment set to REFUNDED; cancellation notice sent with `refunded: true`	As expected	PASS
TC-U-22	Unit	payments	Gateway webhooks are refused in mock mode. Source: `payments.service.spec.ts`, "refuses gateway webhooks in mock mode"	`PAYMENTS_MODE=mock`	Call `handleStripeWebhook`	`BadRequestException`	As expected	PASS
TC-U-23	Unit	subscriptions	Payment failure restock on renewal. Source: `apps/api/src/subscriptions/billing-cron.service.spec.ts`, "restocks, removes the reserved order and records the decline when the charge fails"	Due ACTIVE subscription; charge declines	Run the billing cycle	Reserved stock returned; reserved order removed; decline recorded	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-24	Unit	subscriptions	Zero stock on renewal: nothing is charged. Source: `billing-cron.service.spec.ts`, "charges nothing when stock is short and advances the date"	Stock check reports not enough	Run the billing cycle	No charge; next billing date moved on	As expected	PASS
TC-U-25	Unit	subscriptions	A cycle is never charged twice. Source: `billing-cron.service.spec.ts`, "never charges a cycle twice: a duplicate cycle key only advances the date"	Cycle key already used	Run the billing cycle again	No second charge; date moved on	As expected	PASS
TC-U-26	Unit	subscriptions	Another customer's subscription is refused. Source: `apps/api/src/subscriptions/subscriptions.service.spec.ts`, "returns 403 for another customer's subscription"	Subscription owned by customer A	Customer B calls a transition	403	As expected	PASS
TC-U-27	Unit	orders	A customer can cancel only their own PENDING_PAYMENT order. Source: `apps/api/src/orders/order-status.service.spec.ts`, "customer can self-cancel only their own PENDING_PAYMENT order"	Orders in several states and owners	Try to cancel as the customer	Allowed only for own PENDING_PAYMENT order	As expected	PASS
TC-U-28	Unit	delivery	DELIVERED needs a proof photo. Source: `apps/api/src/delivery/delivery.service.spec.ts`, "rejects DELIVERED without a proof photo (food-safety / chain-of-custody)"	Assignment OUT_FOR_DELIVERY, no photo	Move to DELIVERED	`BadRequestException`	As expected	PASS
TC-U-29	Unit	delivery	A rider cannot act on another rider's assignment. Source: `delivery.service.spec.ts`, "rejects a foreign rider (ownership check)"	Assignment owned by rider A	Rider B updates status	403	As expected	PASS
TC-U-30	Unit	reviews	A review needs a DELIVERED order for the product. Source: `apps/api/src/reviews/reviews.service.spec.ts`, "rejects when customer has no DELIVERED order for the product (anti-fraud)"	No DELIVERED order for the product	Call `create`	Request refused	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-31	Unit	common	UTC midnight: the cron lock key uses the UTC date exactly at midnight. Source: `apps/api/src/common/locks/cron-lock.service.spec.ts`, "utcDateKey uses the UTC calendar date exactly at midnight"	Clock set to 00:00 UTC	Call `utcDateKey`	The new UTC date	As expected	PASS
TC-U-32	Unit	common	One user is limited across several IP addresses. Source: `apps/api/src/common/guards/user-throttler.guard.spec.ts`, "blocks one user across several IP addresses once the default budget is spent"	Default budget used up by one user id	Send from a new IP address	Request blocked	As expected	PASS
TC-U-33	Unit	uploads	A file that is not a real image is refused, whatever its claimed type. Source: `apps/api/src/uploads/uploads.service.spec.ts`, "rejects anything else, whatever its claimed type"	Bytes that are not JPEG, PNG, WebP or HEIC	Call `sniffImageType`	No type detected; upload refused	As expected	PASS
TC-U-34	Unit	config	A short JWT secret and bad numbers are reported. Source: `apps/api/src/config/env.validation.spec.ts`, "rejects a short JWT secret and malformed numeric settings"	Env with a short `JWT_SECRET`	Call `collectEnvProblems`	Problems listed for each bad value	As expected	PASS

4.3 Web and mobile unit tests (Vitest, Flutter)

All 40 Vitest tests passed. `flutter test` was not run in this measurement.

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-35	Unit	storefront	Zero stock in the UI: the stock indicator shows "out of stock" at 0. Source: `apps/storefront/components/stock-indicator.test.tsx`, "shows out of stock at zero"	jsdom	Render `StockIndicator` with 0	Out-of-stock text shown	As expected	PASS
TC-U-36	Unit	storefront	Purchase buttons are disabled when every variant is out of stock. Source: `apps/storefront/app/products/[slug]/variant-panel.test.tsx`, "disables purchase buttons when every variant is out of stock"	All variants at 0 stock	Render `VariantPanel`	Buttons disabled	As expected	PASS
TC-U-37	Unit	storefront	The API client sends the CSRF header on non-GET calls. Source: `apps/storefront/lib/api.test.ts`, "sends the CSRF header and credentials on non-GET calls"	`fetch` mocked; CSRF cookie set	Call `api()` with POST	`x-csrf-token` header and credentials sent	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-U-38	Unit	storefront	Open redirects are refused. Source: `apps/storefront/lib/navigation.test.ts`, "rejects absolute, protocol-relative and backslash redirects"	None	Call `safeNextPath` with unsafe values	Falls back to a safe path	As expected	PASS
TC-U-39	Unit	admin	Fulfilment staff see only the operations pages. Source: `apps/admin/lib/nav.test.ts`, "shows fulfilment staff only the operations pages"	Role FULFILMENT_STAFF	Build the navigation	Only operations pages listed	As expected	PASS
TC-U-40	Unit	admin	Batch status badges use the SDS §6.6 colours. Source: `apps/admin/components/BatchStatusBadge.test.tsx`, "BatchStatusBadge (SDS §6.6 colours)" table test	jsdom	Render each status	Each status has its colour and text	As expected	PASS
TC-U-41	Unit	seller	The seller portal sends visitors to sign in and refuses other roles. Source: `apps/seller/lib/guard.test.ts`, "waits, sends anonymous visitors to sign in and refuses other roles"	None	Call the guard with each session state	Correct redirect or refusal	As expected	PASS
TC-U-42	Unit	ui	The modal traps keyboard focus. Source: `packages/ui/src/components/__tests__/Modal.test.tsx`, "moves focus inside on open and traps Tab within the panel"	jsdom	Open the modal; press Tab repeatedly	Focus stays inside the panel	As expected	PASS
TC-U-43	Unit	delivery app	"Mark delivered" stays disabled until a photo is taken. Source: `apps/delivery/test/proof_of_delivery_screen_test.dart`, "Mark delivered stays disabled until a photo is captured"	Flutter test runner	Pump the screen without a photo	Button disabled	Not run in this measurement	NOT RUN
TC-U-44	Unit	delivery app	A proof photo over 5 MB is refused before the API call. Source: `apps/delivery/test/upload_service_test.dart`, "rejects a photo over 5 MB before calling the API"	Flutter test runner	Upload a file over 5 MB	Error; no API call	Not run in this measurement	NOT RUN

4.4 Integration tests (Supertest)

All 36 integration tests are listed. 31 passed and 5 failed. Each suite boots the full `AppModule` against PostgreSQL and Redis with seeded accounts (`tharushi@example.com`, `rajiv@example.com`, `admin@biobloom.lk`, `kitchen-amma@biobloom.lk`, `rider1@biobloom.lk`, password `Password123!`).

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-01	Integration	health	The health probe is public. Source: `apps/api/test/auth.e2e-spec.ts`, "GET /api/health is public"	App running	`GET /api/health` with no cookie	200, `status: ok`	200, ok	PASS
TC-I-02	Integration	auth	A POST without a CSRF token is refused. Source: `auth.e2e-spec.ts`, "rejects POST without CSRF token"	None	`POST /api/auth/login` without the CSRF header	403	403	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-03	Integration	auth	Full session: register, login, profile, logout, then blocked. Source: `auth.e2e-spec.ts`, "register → login → /me → logout → blocked"	New unique email	1) Register. 2) Login. 3) `GET /api/auth/me`. 4) Logout. 5) `GET /api/auth/me` again.	201; 200 with an HttpOnly, SameSite=Strict cookie; 200; 204; 401	As expected	PASS
TC-I-04	Integration	auth	Unknown email and wrong password give the same 401. Source: `auth.e2e-spec.ts`, "rejects unknown email and wrong password identically (401)"	Seeded customer	Login with an unknown email, then with a wrong password	Both 401 with the same status code	As expected	PASS
TC-I-05	Integration	auth	A bad email in the body gives 422. Source: `auth.e2e-spec.ts`, "validates DTO (422 on bad email)"	None	Register with an invalid email	422	422	PASS
TC-I-06	Integration	checkout	Full checkout in mock payment mode. Source: `apps/api/test/checkout.e2e-spec.ts`, "full checkout: cart → quote → checkout → webhook → PAID"	Own product with 3 batches (2, 3 and 5 units)	1) Add 2 units. 2) Get the quote. 3) Check out. 4) Post a forged Stripe webhook. 5) Simulate payment success. 6) Simulate failure again.	Subtotal 1000.00, 600 g, shipping 500.00, total 1500.00; order PENDING_PAYMENT; earliest batch 0 and DEPLETED; cart empty; webhook 400; order PAID; repeat changes nothing	As expected	PASS
TC-I-07	Integration	checkout	Concurrent checkout race for the last unit. Source: `checkout.e2e-spec.ts`, "concurrent checkout for the last unit: one wins, one 409"	One unit left; two customers each have it in the cart	Send both `POST /api/checkout` calls at the same time	Exactly one 201 and one 409	Statuses [201, 409]	PASS
TC-I-08	Integration	delivery	Admin assigns, rider moves through each status, order becomes DELIVERED. Source: `apps/api/test/delivery.e2e-spec.ts`, "happy path: admin assigns → rider transitions → DELIVERED → Order auto-flips"	Own SHIPPED order; seeded rider `rider1@biobloom.lk`	1) Admin assigns. 2) Rider logs in and registers an FCM token. 3) Rider lists open assignments. 4) Status changes, proof upload, DELIVERED.	Step 3 returns exactly 1 assignment; order later DELIVERED; audit rows written	Failed at step 3: "Expected length: 1, Received length: 2". The second item is the seeded assignment "Customer prefers afternoon delivery."	FAIL
TC-I-09	Integration	delivery	A rider gets 404 for an assignment that does not exist. Source: `delivery.e2e-spec.ts`, "rejects a foreign rider trying to act on another rider's assignment"	Rider logged in	Post a status for a random UUID	404	404	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-10	Integration	subscriptions	The payment token round-trips through pgcrypto. Source: `apps/api/test/subscription-cron.e2e-spec.ts`, "TokenCipherService round-trips through pgcrypto pgp_sym_encrypt/decrypt"	`pgcrypto` extension from the first migration	Encrypt, then decrypt a token	Ciphertext longer than the text; decrypted value equals the original	As expected	PASS
TC-I-11	Integration	subscriptions	Billing cron: only ACTIVE and due subscriptions are billed; zero stock and declined cards are handled; restock after decline. Source: `subscription-cron.e2e-spec.ts`, "selects ACTIVE+due only; skips PAUSED/CANCELLED; advances all touched dates"	Six subscriptions: ACTIVE due, PAUSED, CANCELLED, ACTIVE in 14 days, ACTIVE on a variant with 0 stock, ACTIVE with a declining card; batch of 50	Call `runBilling(today)`	due 3, renewed 1, outOfStock 1, chargeFailed 1; renewed order PAID for 800.00; out-of-stock payment FAILED `out_of_stock`; dates moved on 30 days; batch 49 (declined charge returned its unit)	As expected	PASS
TC-I-12	Integration	refunds	Admin approves a refund; order REFUNDED; fresh batch restocked, expired batch not. Source: `apps/api/test/refund.e2e-spec.ts`, "customer requests, admin approves → Order REFUNDED, batch restocked except for expired"	Own DELIVERED order: 2 units from a fresh batch, 1 from an EXPIRED batch; PAID payment 1850.00	1) Customer requests a refund. 2) Admin approves. 3) Check response, order, payment, batches, audit.	Approve response has `refundRequestId` and `restocked` (2 rows); fresh batch 5, expired stays 3	Failed at line 257: `approveRes.body.refundRequestId` was `undefined`	FAIL
TC-I-13	Integration	refunds	A second refund request on the same order is refused. Source: `refund.e2e-spec.ts`, "rejects a duplicate refund request on the same order"	Order no longer DELIVERED	Request a refund again	400	400	PASS
TC-I-14	Integration	sellers	Seller lifecycle: apply, listed as pending, approved, can log in. Source: `apps/api/test/seller.e2e-spec.ts`, "full lifecycle: apply → list pending → approve → login as seller"	Unique seller email	Apply; try to log in; admin lists and approves; log in	201 PENDING; login 403 before approval; APPROVED with an ADMIN audit row; login role SELLER	As expected	PASS
TC-I-15	Integration	sellers	Rejection saves the reason, writes an audit row and blocks login. Source: `seller.e2e-spec.ts`, "reject path persists reason and audits, blocks login"	New applicant	Admin rejects with "No tax ID provided"; seller tries to log in	REJECTED; reason saved; audit row; login 403	As expected	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-16	Integration	sellers	A duplicate application gives 409. Source: `seller.e2e-spec.ts`, "duplicate apply returns 409"	Email already used	Apply again	409	409	PASS
TC-I-17	Integration	sellers	Admin routes need a signed-in admin. Source: `seller.e2e-spec.ts`, "admin endpoints require an authenticated admin"	No cookie	`GET /api/admin/sellers/pending`	401	401	PASS
TC-I-18	Integration	sellers	A customer gets 403 on admin routes. Source: `seller.e2e-spec.ts`, "non-admin (customer) gets 403 on admin endpoints"	Customer session	Call an admin route	403	403	PASS
TC-I-19	Integration	inventory	Expired batch: available stock leaves out expired batches. Source: `apps/api/test/fifo.e2e-spec.ts`, "availableStock sums only valid batches (excludes expired)"	Batches: EXPIRED 50 units (5 days ago), 5 units (15 days), 7 units (60 days), 10 units (120 days)	Call `availableStock`	22	22	PASS
TC-I-20	Integration	inventory	Stock check at the exact boundary. Source: `fifo.e2e-spec.ts`, "checkStock returns sufficient=true / false at the boundary"	22 valid units	`checkStock` for 22, then 23	true for 22, false for 23	As expected	PASS
TC-I-21	Integration	inventory	FIFO takes the earliest expiry first. Source: `fifo.e2e-spec.ts`, "FIFO decrement: takes earliest-expiry batch first"	Same batches	Decrement 3	Early batch 5 to 2, still ACTIVE; middle batch untouched	As expected	PASS
TC-I-22	Integration	inventory	FIFO spans batches and marks drained ones DEPLETED. Source: `fifo.e2e-spec.ts`, "FIFO decrement spans batches and marks DEPLETED when drained"	After TC-I-21	Decrement across early and middle batches	Early and middle 0 and DEPLETED; late batch 10, ACTIVE	As expected	PASS
TC-I-23	Integration	inventory	Expired batch: never used, even when valid stock is short. Source: `fifo.e2e-spec.ts`, "NEVER touches an expired batch even if the union is short"	10 valid units, 50 expired	Decrement 12 in a transaction	`OutOfStockError`; expired batch still 50 and EXPIRED	As expected	PASS
TC-I-24	Integration	products	Zero stock: a product with no valid stock is hidden from the catalogue. Source: `fifo.e2e-spec.ts`, "GET /api/products excludes products with zero valid stock"	Drain the last 10 valid units	Call `ProductsService.list` (the service behind `GET /api/products`)	Product not in the list	Not in the list	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-25	Integration	payments	A Stripe event with a tampered body is refused and the order stays unpaid. Source: `apps/api/test/webhook-signature.e2e-spec.ts`, "rejects a tampered body and leaves the order unpaid"	Own order and PENDING payment; the suite tries to switch to live mode	Sign one body, send another	400; order PENDING_PAYMENT	400; order PENDING_PAYMENT. Analysis (Section 5.1): the 400 came from the mock-mode guard, not the signature check	PASS
TC-I-26	Integration	payments	A replayed Stripe event outside the 300 s tolerance is refused. Source: `webhook-signature.e2e-spec.ts`, "rejects a replayed event outside the timestamp tolerance"	As above	Sign with a timestamp 1 hour old	400	400. Same caveat as TC-I-25	PASS
TC-I-27	Integration	payments	A correctly signed success is applied once; the repeat is a duplicate. Source: `webhook-signature.e2e-spec.ts`, "applies a correctly signed success once, then reports duplicates"	As above	Send a valid signed `checkout.session.completed` twice	200 `applied`, order PAID; then 200 `duplicate`	"expected 200 "OK", got 400 "Bad Request"" at line 122	FAIL
TC-I-28	Integration	payments	A signed event type we do not handle is acknowledged. Source: `webhook-signature.e2e-spec.ts`, "acknowledges event types it does not handle"	As above	Send a valid signed `customer.created`	200 `ignored`	"expected 200 "OK", got 400 "Bad Request"" at line 139	FAIL
TC-I-29	Integration	orders	Another customer's order is refused. Source: `apps/api/test/ownership.e2e-spec.ts`, "refuses another customer's order"	Order of customer A; session of customer B	View, cancel and refund-request the order	403 each time	403	PASS
TC-I-30	Integration	subscriptions	Another customer's subscription is refused. Source: `ownership.e2e-spec.ts`, "refuses another customer's subscription"	As above	Pause and cancel	403	403	PASS
TC-I-31	Integration	checkout	Another customer's address is refused at checkout. Source: `ownership.e2e-spec.ts`, "refuses another customer's address at checkout"	As above	Check out with A's address id	400 or 403	Within [400, 403]	PASS
TC-I-32	Integration	cart	Another customer's cart line is refused. Source: `ownership.e2e-spec.ts`, "refuses another customer's cart line"	As above	Update and delete A's cart line	403	403	PASS

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-I-33	Integration	support	Another customer's support ticket is refused. Source: `ownership.e2e-spec.ts`, "refuses another customer's support ticket"	As above	Reopen A's ticket	403	403	PASS
TC-I-34	Integration	shipping	Weight bracket edge through the API: 500 g. Source: `apps/api/test/shipping-quote.e2e-spec.ts`, "%i g is charged %s" row 500	Own 500 g variant with stock; seeded brackets	Add one to the cart; `GET /api/checkout/quote`	`totalWeightG` 500, `shippingFee` 350.00	500, 350.00	PASS
TC-I-35	Integration	shipping	Weight bracket edge through the API: 501 g. Source: `shipping-quote.e2e-spec.ts`, row 501	Own 501 g variant	Same steps	`totalWeightG` 501, `shippingFee` 500.00	501, 500.00	PASS
TC-I-36	Integration	notifications	All 8 SDS §4.2 email templates reach MailHog. Source: `apps/api/test/notifications.e2e-spec.ts`, "emits all 8 SDS §4.2 templates and MailHog receives matching messages"	MailHog on port 8025; inbox cleared	Emit the 8 events; poll MailHog for the `X-Biobloom-Template` header	All 8 templates received	7 of 8 received; `d-batch-expiring-soon` missing	FAIL

4.5 Browser end-to-end tests (Playwright)

These suites need the full stack running. They are not part of `ci.yml` and were not run in this measurement.

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-E-01	E2E	storefront	A customer buys a product from start to finish. Source: `apps/storefront/e2e/checkout.spec.ts`, "a customer can buy a product end to end"	Stack up and seeded; mock payments	Sign in, open a product, add to cart, check out, pay on the mock page	"Thank you for your order!" heading	Not run	NOT RUN
TC-E-02	E2E	storefront	An anonymous visitor is sent to sign in before checkout. Source: `checkout.spec.ts`, "an anonymous visitor is sent to sign in before checkout"	Stack up	Open `/checkout`	Redirect to `/login?next=%2Fcheckout`	Not run	NOT RUN
TC-E-03	E2E	admin	An admin views the dashboard and adds a batch. Source: `apps/admin/e2e/batches.spec.ts`, "an admin signs in, views the dashboard and adds a batch"	Stack up	Sign in, open a product, add a batch	Batch appears	Not run	NOT RUN
TC-E-04	E2E	admin	Expired batch at the UI: a past expiry date shows the API error. Source: `batches.spec.ts`, "a past expiry date shows the API error beside the form"	Stack up	Enter expiry 2020-01-01 and submit	Alert mentioning expiry	Not run	NOT RUN

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-E-05	E2E	seller	A seller creates a product and its first batch. Source: `apps/seller/e2e/products.spec.ts`, "a seller creates a product and adds its first batch"	Stack up; approved seller	Create product, add batch	Product and batch saved	Not run	NOT RUN
TC-E-06	E2E	storefront	No serious accessibility violations on key pages. Source: `apps/storefront/e2e/a11y.spec.ts`, "no serious accessibility violations on ${path}"	Stack up	Open each page and run axe	No serious or critical violations	Not run	NOT RUN

4.6 Requested boundary cases

Boundary case	Covered by (status)	Not yet covered
Expired batches	TC-U-16 (PASS), TC-I-19 (PASS), TC-I-23 (PASS), TC-E-04 (NOT RUN)	Batch expiring exactly today (TC-G-01); refund restock of an expired batch has no passing test (TC-G-05)
Zero stock	TC-U-06, TC-U-08, TC-U-24, TC-U-35, TC-U-36, TC-I-11, TC-I-24 (all PASS)	Add to cart above valid stock (TC-G-04)
Weight bracket edges, 500 g vs 501 g	TC-U-10, TC-U-11 (PASS); TC-I-34, TC-I-35 (PASS). Seed brackets: `packages/prisma/prisma/seed.ts` lines 37 to 41	None for 500/501. Other edges are unit-only (TC-U-12, TC-U-13)
Payment failure restock	TC-U-18 (PASS), TC-U-23 (PASS), TC-I-11 (PASS, subscription renewal)	One-off order failure through the API (TC-G-03)
Concurrent checkout race	TC-I-07 (PASS); TC-U-19 (PASS) for the webhook claim	None

4.7 Gap cases: no automated test yet

These cases are needed by DDP-#33 AC3, DDP-#34 or the SDS, but no automated test exists today. All are NOT RUN.

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-G-01	Unit	inventory, batches	A batch whose expiry date is today stays sellable today and is not marked EXPIRED. Code: `inventory.service.ts` line 61 (`gte: startOfTodayUtc()`), `expiry-cron.service.ts` line 65 (`lt: today`)	Clock at 00:00 UTC; batch expiry = today	Call `availableStock` and `sweep`	Stock counted; batch not EXPIRED	No automated test yet	NOT RUN
TC-G-02	Unit	batches	Expiring-soon window edge: day 7 is flagged, day 8 is not. Code: `expiry-cron.service.ts` line 97 (`lte: sevenDaysOut`)	Batches at +7 and +8 days	Call `sweep`	Only the +7 batch is EXPIRING_SOON	No automated test yet	NOT RUN
TC-G-03	Integration	payments, inventory	One-off order payment failure returns stock through the API. Code: `payments.service.ts` lines 143 to 164 (`simulate`)	Order PENDING_PAYMENT after checkout	`POST /api/payments/:id/simulate` with `failure`	Order CANCELLED; batch quantities restored	No automated test yet (unit only, TC-U-18)	NOT RUN

Test Case ID	Type	Module	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-G-04	Integration	cart	Adding more than the valid stock is refused. Code: `cart.service.ts` lines 287 to 290	Variant with 0 valid units	`POST /api/cart/items`	409	No automated test yet; `cart` has 0% coverage	NOT RUN
TC-G-05	Unit	inventory	Refund restock skips an expired batch. Code: `inventory.service.ts` line 172	Order item on an EXPIRED batch	Call `restockOrderItems`	`restocked: false`; quantity unchanged	Only TC-I-12 checks this, and it fails before the check	NOT RUN
TC-G-06	Integration	payments	Stripe signature, replay window and success are verified in live mode. Code: `stripe.gateway.ts` lines 34 to 49	Payments config really in live mode	Tampered, replayed and valid events	400, 400, 200 `applied`	TC-I-25 and TC-I-26 pass only because of the mock-mode guard; no unit test calls `verifyStripeWebhook`	NOT RUN
TC-G-07	Unit	payments	A PayHere notification with a bad signature is refused. Code: `payhere.gateway.ts` line 64 (`verifyPayHereNotification`)	Merchant secret set	Verify a notification with a wrong `md5sig`	Error	No automated test yet	NOT RUN
TC-G-08	Unit	promo-codes	A promo code at its expiry time and at the minimum order amount. Code: `promo-codes.service.ts` lines 49 and 53	Code with `validUntil` and `minOrderLkr`	Apply just before and after expiry; with subtotal equal to and below the minimum	Accepted, then 422	No automated test yet; `promo-codes` has 0% coverage	NOT RUN

4.8 Register summary

Type	PASS	FAIL	NOT RUN	Total
Unit	42	0	7	49
Integration	31	5	3	39
E2E	0	0	6	6
Total	**73**	**5**	**16**	**94**

NOT RUN breaks down as: 2 Flutter tests and 6 Playwright tests that exist but were not run, and 8 gap cases with no automated test yet.

---

5. Analysis of the failing integration tests

> **Analysis.** This section is our reading of the test code and the product code. No code was changed. Each cause is the most likely one based on the evidence shown. It still needs to be confirmed by the fix.

Test	Classification	Product bug?
TC-I-27, TC-I-28 (and false passes in TC-I-25, TC-I-26)	Test harness defect: env set after the config snapshot	No evidence of one
TC-I-08	Test data assumption: rider already has a seeded open assignment	No
TC-I-12	Test out of date: API response contract changed	No (contract change)

Test	Classification	Product bug?
TC-I-36	Test data defect: non-UUID id in the event payload	No evidence of one

5.1 Webhook signature suite (TC-I-27, TC-I-28)

Symptom. Both correctly signed events get `400 Bad Request` (lines 122 and 139 of `apps/api/test/webhook-signature.e2e-spec.ts`).

Evidence.

1. The suite sets `PAYMENTS_MODE=live` and the Stripe secrets inside `beforeAll` (lines 23 to 35). But the file imports `./helpers/app`, which imports `AppModule` at load time.
2. `AppModule` calls `ConfigModule.forRoot({ validate: validateEnv })` (`apps/api/src/app.module.ts` lines 46 to 52). In `@nestjs/config` 3.3.0, `forRoot` copies `process.env` and runs `validate` straight away, during the import. `validateEnv` returns that copy (`apps/api/src/config/env.validation.ts` lines 136 to 147).
3. `ConfigService.get` reads the validated copy before `process.env`. So `PAYMENTS_MODE` resolves to the CI value `mock`, not `live`.
4. In mock mode, `handleStripeWebhook` throws `BadRequestException('Stripe webhooks are not enabled in mock payment mode')` before it checks the signature (`apps/api/src/payments/payments.service.ts` lines 212 to 216). The unit test TC-U-22 confirms this guard.

Most likely cause. The suite switches to live mode too late, so the API stays in mock mode and refuses every webhook.

Side effect. TC-I-25 (tampered body) and TC-I-26 (replay) pass, but for the wrong reason. Their 400 also comes from the mock-mode guard. So no passing test today proves the Stripe signature and replay checks in `stripe.gateway.ts` lines 34 to 49. The DDP-#34 negative test ("a webhook with a tampered signature returns 400") is therefore only formally met.

Suggested fix (not applied). Set the live-mode variables before `AppModule` is imported, for example in a per-suite setup file or with a dynamic import after `Object.assign`. Another option is to override the `PaymentGatewayRegistry` provider in the testing module. Then make the tampered test assert the error message, so it cannot pass for another reason.

5.2 Delivery happy path (TC-I-08)

Symptom. `GET /api/delivery/assignments` returns 2 assignments. The test expects 1 (line 239 of `apps/api/test/delivery.e2e-spec.ts`).

Evidence.

1. The seed gives `rider1@biobloom.lk` an assignment with status `OUT_FOR_DELIVERY` and the note "Customer prefers afternoon delivery." (`packages/prisma/prisma/seed.ts` lines 1083 to 1097). This is the extra item in the failure output.
2. `listForRider` returns every assignment in `ASSIGNED`, `PICKED_UP` or `OUT_FOR_DELIVERY` (`apps/api/src/delivery/delivery.service.ts` lines 52 to 56 and 179 onward). Two open assignments is the correct answer.
3. The failure happens on a fresh database too, because the seed always creates this assignment.

Most likely cause. Test data assumption. The test was written before the seed gave the rider a demo assignment.

Side effect. The rest of the happy path (status changes, proof upload, order cascade to DELIVERED, audit rows, admin proof view) is not run at integration level. Only the unit tests TC-U-28 and TC-U-29 cover parts of it.

Suggested fix (not applied). Find the new assignment by id in the list instead of checking the list length. Another option is a dedicated rider created by the suite.

5.3 Refund approval (TC-I-12)

Symptom. `approveRes.body.refundRequestId` is `undefined` (line 257 of `apps/api/test/refund.e2e-spec.ts`).

Evidence.

1. `RefundsService.approve` now returns the refund request row: `return this.prisma.refundRequest.findUnique(...)` (`apps/api/src/refunds/refunds.service.ts` line 221). The row has `id` and `status`. It has no `refundRequestId` or `restocked` fields.
2. The per-batch restock result still exists. It is only written to the audit metadata (`refunds.service.ts` lines 201 to 212).
3. The final status is `PROCESSED` (`refunds.service.ts` line 178). The test expects `APPROVED` at line 270, so it would fail there next.

4. `docs/api-reference.md` line 32 lists the route but does not define the response body. So the test, not a document, holds the old contract.

Most likely cause. The test is out of date. The approve flow was reworked (partial refunds, a `restock` option, a `PROCESSED` state), and the response shape changed. The test was not updated.

Side effect. The food-safety rule "do not restock an expired batch" (`inventory.service.ts` line 172) has no passing test at any level (gap TC-G-05).

Suggested fix (not applied). Agree on the response contract, then update the test. It should read the batch quantities and the audit metadata, and expect `PROCESSED`. If admins need the restock list in the UI, return it from the API and document it in `docs/api-reference.md`.

5.4 Notifications dispatch (TC-I-36)

Symptom. MailHog received 7 of the 8 templates. `d-batch-expiring-soon` is missing.

Evidence.

1. The test emits `batch.expiringSoon` with `productVariantId: 'v-1'` (`apps/api/test/notifications.e2e-spec.ts` lines 131 to 138).
2. The listener first calls `batchContext(p.productVariantId)`, which runs `prisma.productVariant.findUnique({ where: { id } })` (`apps/api/src/notifications/notifications.service.ts` lines 459 to 460 and 656 to 666).
3. `ProductVariant.id` is a `@db.Uuid` column (`packages/prisma/prisma/schema.prisma` line 189). `'v-1'` is not a valid UUID, so the query throws an error.
4. The listener wrapper catches the error and only logs `Listener batch.expiringSoon failed: ...` (`notifications.service.ts` lines 553 to 561). No email is sent.
5. The other 7 events arrived, so SMTP delivery and the mailer work. Only the listener that looks up a variant by this id fails.

Most likely cause. Test data defect. A valid UUID, even one that does not exist, would make `batchContext` return no variant. The email would then still go to the admins.

Product note. The real expiry cron passes real batch and variant ids (`apps/api/src/batches/expiry-cron.service.ts` lines 118 to 126), so we found no product bug. The listener does swallow errors silently, though, which makes this kind of fault hard to see. A metric or an error-level alert on listener failures would help.

Suggested fix (not applied). Use the id of a seeded variant in the payload.

---

6. Part B: Coverage Report

6.1 Executive summary

- Unit coverage of the API is **31.24% statements, 23.38% branches, 21.82% functions, 31.54% lines**.
- The gate in `apps/api/jest.config.json` is **65 / 50 / 65 / 65** (statements, branches, functions, lines). It is **not met** on any metric, so `test:cov` exits non-zero.
- The Definition of Done target of **80%** is also not met.
- 10 of the 29 API modules have 0% coverage: addresses, analytics, bundles, cart, fulfilment, health, products, promo-codes, refunds and staff. `redis` is close behind at 13.3% lines.
- Only `auth` (80.4% lines) and `i18n` (93.2% lines) reach 80%.
- The web unit tests all pass. Web coverage was not measured.

6.2 Global numbers against the gate and the target

Metric	Covered / total	Measured	Gate (`jest.config.json`)	Gap to gate	Target (DoD)	Gap to 80%
Statements	1198 / 3834	**31.24%**	65%	33.76 points, 1295 more statements	80%	48.76 points, 1870 more statements
Branches	348 / 1488	**23.38%**	50%	26.62 points, 396 more branches	80%	56.62 points, 843 more branches

Metric	Covered / total	Measured	Gate (`jest.config.json`)	Gap to gate	Target (DoD)	Gap to 80%
Functions	163 / 747	**21.82%**	65%	43.18 points, 323 more functions	80%	58.18 points, 435 more functions
Lines	1085 / 3440	**31.54%**	65%	33.46 points, 1151 more lines	80%	48.46 points, 1667 more lines

The "more" columns are the extra items that must be covered for the metric to reach the threshold, with today's totals.

Note on scope: `collectCoverageFrom` leaves out `*.module.ts`, `*.dto.ts`, `dto/**`, `*.d.ts` and `main.ts`. It **includes controllers**. Controllers are mostly covered by the Supertest suites, which do not count towards Jest unit coverage.

6.3 Coverage matrix per API module

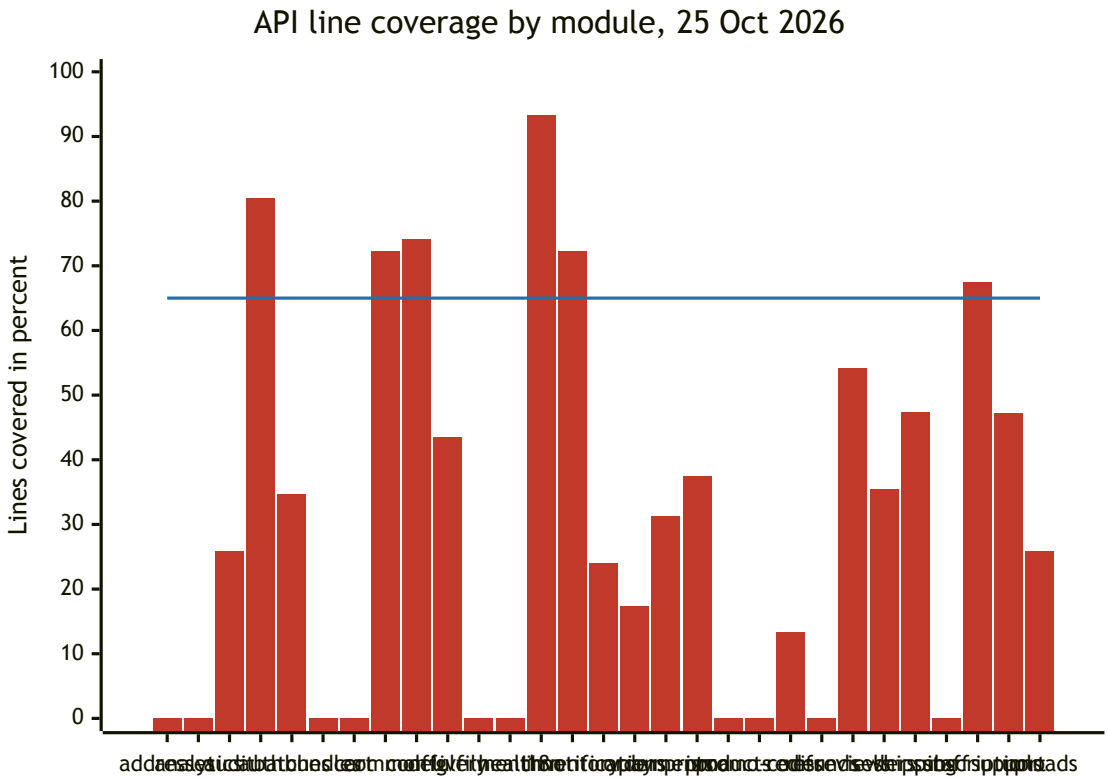
Source: the Jest coverage run on 25 October 2026. "Lines total" is the number of executable lines. "Lines not covered" is lines total multiplied by (100% minus line coverage), rounded.

Module	Statements %	Branches %	Functions %	Lines %	Lines total	Lines not covered	Meets 65% lines
addresses	0.0	0.0	0.0	0.0	33	33	No
analytics	0.0	0.0	0.0	0.0	55	55	No
audit	30.0	0.0	0.0	25.9	27	20	No
auth	81.4	84.5	70.4	80.4	189	37	Yes
batches	33.0	8.0	27.8	34.7	98	64	No
bundles	0.0	0.0	0.0	0.0	156	156	No
cart	0.0	0.0	0.0	0.0	140	140	No
common	71.9	55.8	81.0	72.3	119	33	Yes
config	75.0	85.0	75.0	74.1	81	21	Yes
delivery	42.2	48.1	31.3	43.4	182	103	No
fulfilment	0.0	0.0	0.0	0.0	49	49	No
health	0.0	0.0	0.0	0.0	18	18	No
i18n	92.7	74.1	90.0	93.2	44	3	Yes
inventory	71.4	47.4	77.8	72.3	65	18	Yes
notifications	24.9	12.6	16.0	23.9	251	191	No
orders	17.5	16.3	11.3	17.3	271	224	No
payments	29.9	14.4	22.0	31.3	416	286	No
prisma	50.0	n/a	0.0	37.5	8	5	No
products	0.0	0.0	0.0	0.0	313	313	No
promo-codes	0.0	0.0	0.0	0.0	67	67	No
redis	18.2	0.0	0.0	13.3	30	26	No
refunds	0.0	0.0	0.0	0.0	105	105	No
reviews	53.6	31.3	28.6	54.2	59	27	No
sellers	33.3	12.0	18.8	35.4	127	82	No
shipping	48.2	44.0	37.0	47.3	93	49	No
staff	0.0	0.0	0.0	0.0	56	56	No
subscriptions	64.9	50.8	54.3	67.5	246	80	Yes
support	45.9	54.5	28.6	47.2	53	28	No
uploads	28.0	37.7	15.4	25.8	89	66	No
All files	**31.24**	**23.38**	**21.82**	**31.54**	**3440**	**2355**	**No**

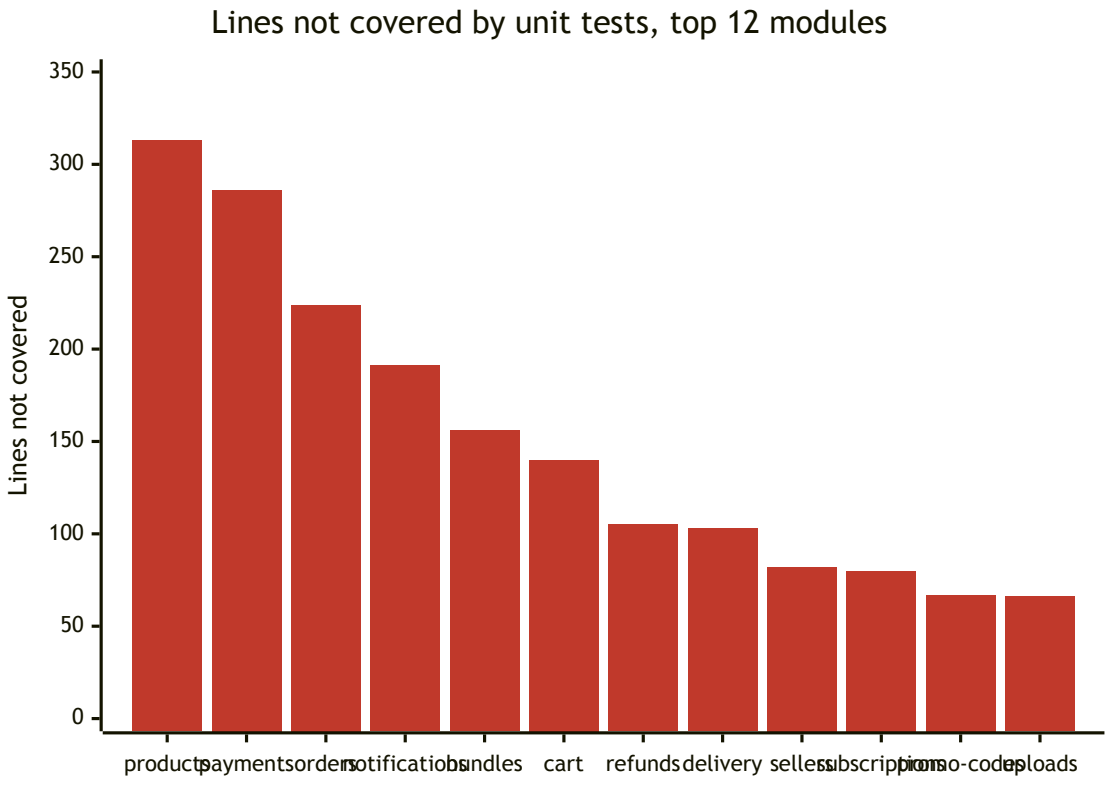
The gate is global, not per module. The last column only shows which modules already pull their weight.

6.4 Charts

Line coverage by module, with the 65% gate drawn as a line:



Lines not covered, the twelve largest modules:



6.5 Web unit tests (Vitest)

Workspace	Files	Tests	Passed	Failed	Coverage
`apps/storefront`	5	11	11	0	Not measured
`apps/admin`	2	7	7	0	Not measured
`apps/seller`	1	2	2	0	Not measured
`packages/ui`	3	20	20	0	Not measured
Total	**11**	**40**	**40**	**0**	Not measured

Each web workspace has a `test:coverage` script (`vitest run --coverage`), but the CI job runs only `test` and has no web coverage gate.

6.6 Team backend repository (separate figure)

This figure is for the team polyrepo `seng34213-biobloom/backend`, not the monorepo. It was measured separately. Do not add it to, or compare it directly with, the monorepo numbers above.

Item	Value
Repository and commit	`seng34213-biobloom/backend`, branch `develop`, commit `55e9ebb`
Unit tests	338 pass
Statement coverage	91.27%
Gate	80%, set in that repository
Scope of that gate	Excludes controllers, DTOs and modules

The two figures differ mainly because they measure different code bases and different scopes. The team repository leaves controllers out of its coverage scope. The monorepo includes them.

---

7. Supertest integration suites

7.1 What each suite covers

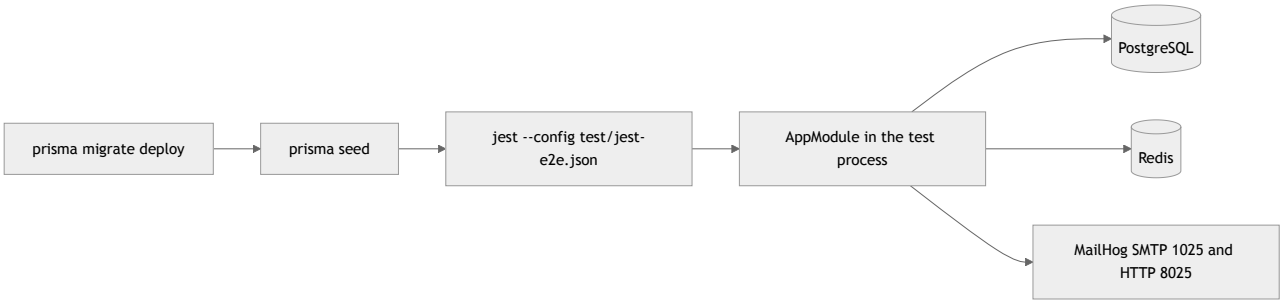
Suite file (`apps/api/test`)	Tests	Result	What it covers	HTTP setup	Time (s)
`auth.e2e-spec.ts`	5	5 pass	Public health route, CSRF refusal, full session with cookie flags, uniform 401, 422 validation	Own setup	1.45
`checkout.e2e-spec.ts`	2	2 pass	Cart, quote, checkout, FIFO decrement, mock payment, idempotent repeat; concurrent last-unit race	Own setup with raw body	5.81
`delivery.e2e-spec.ts`	2	1 pass, 1 fail	Admin assigns, rider status flow, proof photo, order cascade, audit rows; unknown assignment 404	Own setup	1.61
`fifo.e2e-spec.ts`	6	6 pass	`InventoryService` against real Postgres: expired batches left out, boundary stock check, FIFO order, DEPLETED marking, zero-stock catalogue	Own setup	0.47
`notifications.e2e-spec.ts`	1	1 fail	8 email templates delivered to MailHog, read through its HTTP API on port 8025	Default Nest app	8.87
`ownership.e2e-spec.ts`	5	5 pass	403 on another customer's order, subscription, address, cart line and support ticket	`createE2eApp`	0.65
`refund.e2e-spec.ts`	2	1 pass, 1 fail	Refund request, admin approval, restock rules, duplicate request refused	Own setup	1.55
`seller.e2e-spec.ts`	5	5 pass	Seller apply, approve, reject, duplicate, admin-only access	Own setup	2.44
`shipping-quote.e2e-spec.ts`	2	2 pass	Bracket edges 500 g and 501 g through `GET /api/checkout/quote`	`createE2eApp`	0.72

Suite file (`apps/api/test`)	Tests	Result	What it covers	HTTP setup	Time (s)
`subscription-cron.e2e-spec.ts`	2	2 pass	pgcrypto token round trip; billing cron selection, renewal, out-of-stock, decline and restock	Own setup	0.72
`webhook-signature.e2e-spec.ts`	4	2 pass, 2 fail	Stripe signature, replay window, apply once, ignored event types	`createE2eApp`	0.51
Total	**36**	**31 pass, 5 fail**			

Three suites use the shared helper `createE2eApp` in `apps/api/test/helpers/app.ts`. It calls `configureHttpApp` from `apps/api/src/config/http-app.ts`, so the suite runs the same middleware chain as production (helmet, version alias, raw-body capture, cookie parser, validation). The older suites build their own smaller setup. Moving them to the helper would make the tests closer to production.

7.2 How the suites run

Config: `apps/api/test/jest-e2e.json`. Suites run one at a time (`maxWorkers: 1`) with a 30 second timeout. `test/helpers/setup-tz.ts` pins `TZ=UTC` so date edges behave the same on every machine. Each suite boots the full `AppModule`, raises the rate limits for the test, creates its own uniquely named products and orders, and deletes them in `afterAll`.



In CI (`.github/workflows/ci.yml`, job `integration`, lines 77 to 132):

1. Service containers: `postgres:15-alpine`, `redis:7-alpine` and `mailhog/mailhog:latest`, each with a health check.
2. Job environment: `NODE_ENV=test`, `TZ=UTC`, `DATABASE_URL`, `REDIS_URL`, SMTP on `localhost:1025`, `PAYMENTS_MODE=mock`, and test-only values for `JWT_SECRET`, `SUBSCRIPTION_TOKEN_ENC_KEY` and S3.
3. Steps: install with a frozen lockfile, generate the Prisma client, `pnpm --filter @biobloom/prisma migrate:deploy`, `pnpm --filter @biobloom/prisma seed`, then `pnpm --filter @biobloom/api test:e2e`.
4. The first migration creates `pgcrypto` (`packages/prisma/prisma/migrations/20260516033651_initial/migration.sql` line 6), so a fresh container needs nothing else.

Locally:

```
docker compose up -d postgres redis mailhog
pnpm --filter @biobloom/prisma migrate:deploy
pnpm --filter @biobloom/prisma seed
pnpm --filter @biobloom/api test:e2e
```

The measured run used local PostgreSQL 16, Redis 7 and MailHog v1.0.1 with the CI job's environment variables, instead of Docker.

---

8. CI status

Repository: `github.com/ovindu17/biobloom`. Latest CI run: 39751628593, on `main`, 22 October 2026.

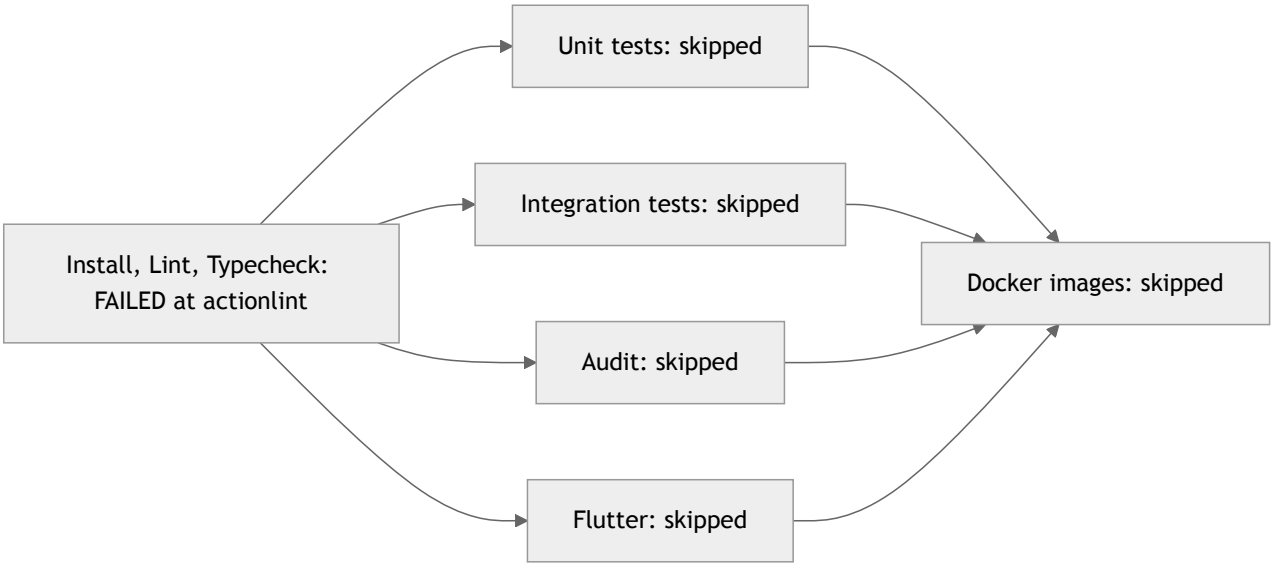
Job (`ci.yml`)	Result in the latest runs	Reason
Install · Lint · Typecheck	**Failed**	Install, Prisma generate, lint, typecheck and Prettier all passed. The `actionlint` step failed.

Job (`ci.yml`)	Result in the latest runs	Reason
Unit tests	Skipped	`needs: lint`
API integration tests	Skipped	`needs: lint`
Dependency audit	Skipped	`needs: lint`
Delivery app (Flutter)	Skipped	`needs: lint`
Docker images, ECR push	Skipped	Need the jobs above

Finding. actionlint runs shellcheck on workflow scripts. It reports `SC2034: i appears unused` for `.github/workflows/deploy.yml` line 90, in the health-check loop:

```
for i in $(seq 1 12); do
  if curl -fsS "$HEALTH_URL" > /dev/null; then echo "Healthy"; exit 0; fi
  sleep 10
done
```

The loop variable is never used. A one-line change (use `_` as the loop variable) should clear the warning. Until it is fixed, CI does not run the unit gate, the integration suites or the audit, so none of the results in this report are checked on each commit.



When the lint job is fixed, we expect, from the local measurement, that these jobs would also fail: `unit` (coverage gate not met), `integration` (5 failing tests) and `audit` (`pnpm audit --audit-level=high` reported 69 high and 10 critical advisories on 25 October 2026).

---

9. Plan to reach the coverage gate

9.1 Priorities

Modules are ranked by lines not covered, then by risk to money, stock and access. The "lines to 80%" column is how many more lines each module must cover to reach 80% line coverage on its own.

Priority	Module	Lines not covered	Lines to 80%	Running total	Why first	First tests to write
1	products	313	250	250	Catalogue and zero-stock filtering; 0% today	List filters, slug lookup, inactive and zero-stock hiding, seller ownership

Priority	Module	Lines not covered	Lines to 80%	Running total	Why first	First tests to write
2	payments	286	203	453	Money; signature checks have no passing test	`verifyStripeWebhook` (tampered, replay, valid), `verifyPayHereNotification`, `parseStripeEvent`, `simulate` failure
3	orders	224	170	623	Checkout, the heart of FIFO and stock	`OrdersService.checkout` happy path, out of stock 409, promo, address ownership
4	notifications	191	141	764	Every customer email; silent listener failures	Each listener with real and missing context; error logging
5	bundles	156	125	889	0% today; stock across bundle items	Bundle availability, price, inactive item
6	cart	140	112	1001	0% today; stock check on add (TC-G-04)	Add above stock 409, merge lines, ownership
7	refunds	105	84	1085	Money and restock rules (TC-G-05)	Full and partial approve, gateway failure rollback, expired batch not restocked
8	delivery	103	67	1152	Order cascade and proof of delivery	`listForRider`, failure reasons, attempts
9	sellers, subscriptions, promo-codes, uploads, batches, staff, analytics, shipping, fulfilment, addresses and the rest	1203	521	1673	Needed for the 80% target	Promo window (TC-G-08), batch expiry edges (TC-G-01, TC-G-02), staff and analytics services

What the numbers show:

- **Priorities 1 to 8 reach the 65% line gate.** Bringing these eight modules to 80% line coverage adds about 1152 covered lines. The gate needs 1151. They are also the eight largest gaps.
- **All modules at 80% reach the 80% target.** That adds about 1673 lines. The target needs 1667.
- **Functions are the furthest behind** (21.82% against 65%, 323 more functions). Many uncovered functions are controller handlers and small helpers. The new suites must call each public method, not just the main path.
- **Branches** need 396 more covered branches for 50%. The per-module data has branch percentages but not branch counts, so we cannot rank modules by branches. The edge cases in Section 4.7 add branches in the riskiest code.

9.2 Working rules

1. Fix the CI lint job first (Section 8), so every new test is checked on each push.
2. Fix the four test defects in Section 5 next. They are small and they bring back real checks: signature verification, the delivery cascade, the refund restock rule and the expiring-soon email.
3. Follow the existing unit test style. Plain objects stand in for `PrismaService`, and `$transaction` passes the callback straight through. Use `jest.useFakeTimers().setSystemTime(...)` for date edges.
4. Add each gap case from Section 4.7 as a named test, so the register can be re-run and updated.
5. Decide, and record in the DDP-#33 pull request, whether controllers stay in the unit coverage scope. The team repository excludes them. If the monorepo does the same, the scope must be stated next to the number.
6. Add `vitest run --coverage` to the web unit job, so web coverage is measured.

---

10. Acceptance check for DDP-#33 and DDP-#34

DDP-#33: Raise API unit-test coverage to the agreed gate

- [x] A global `coverageThreshold` block exists (`apps/api/jest.config.json`: 65/50/65/65).
- [ ] AC1: every suite passes **and** global coverage meets the threshold. Suites pass (231 of 231); coverage does not (31.24% statements).

- [x] AC2: Jest exits non-zero when coverage is below the threshold. The current run is an example.
- [ ] AC3: boundary cases for zero stock, expired batch, bracket edge, UTC midnight and resource ownership in **every** new suite. These exist for inventory, shipping, cron lock and ownership; the new suites for cart, orders, products, batches, refunds and analytics are not written yet.
- [ ] Definition of Done: unit tests passing with 80% coverage or more. Not met.
- [ ] Definition of Done: CI pipeline passes. Not met (Section 8).

DDP-#34: Run the Supertest integration suite in CI

- [x] CI job with PostgreSQL 15, Redis 7 and MailHog service containers, migrations and seed (`ci.yml` lines 77 to 132).
- [ ] AC1: all suites pass. Not met: delivery, notifications and refund fail, and so does the new webhook-signature suite.
- [x] AC2: a failed migration stops the job before the suites (separate "Apply migrations" step).
- [ ] AC3: repeat runs give the same result. The failures repeat on a fresh database, so they are stable. But the suites do not pass yet.
- [x] Shipping-bracket boundary through `GET /api/checkout/quote` added (TC-I-34, TC-I-35, both PASS).
- [ ] Negative test: a tampered signature returns 400 and leaves the order unpaid. The test passes, but only through the mock-mode guard (Section 5.1).
- [ ] The integration job has run green in CI. Not yet: the job is skipped because the lint job fails.

---

11. Evidence index

Evidence	Location
Jest unit config and coverage gate	`apps/api/jest.config.json`
Jest integration config	`apps/api/test/jest-e2e.json`, `apps/api/test/helpers/setup-tz.ts`, `apps/api/test/helpers/app.ts`
API unit specs	`apps/api/src/**/*.spec.ts` (24 files)
API integration suites	`apps/api/test/*.e2e-spec.ts` (11 files)
Web unit tests	`apps/storefront/**/*.test.ts(x)`, `apps/admin/**/*.test.ts(x)`, `apps/seller/lib/guard.test.ts`, `packages/ui/src/components/__tests__/*.test.tsx`
Browser tests	`apps/storefront/e2e`, `apps/admin/e2e`, `apps/seller/e2e`, `apps/*/playwright.config.ts`
Mobile tests	`apps/delivery/test`
CI pipeline	`.github/workflows/ci.yml`; actionlint finding at `.github/workflows/deploy.yml` line 90
Shipping brackets in the seed	`packages/prisma/prisma/seed.ts` lines 37 to 41
Seeded rider assignment (TC-I-08)	`packages/prisma/prisma/seed.ts` lines 1083 to 1097
Config snapshot (TC-I-27, TC-I-28)	`apps/api/src/app.module.ts` lines 46 to 52; `apps/api/src/config/env.validation.ts` lines 136 to 147; `apps/api/src/payments/payments.service.ts` lines 212 to 216
Refund approve response (TC-I-12)	`apps/api/src/refunds/refunds.service.ts` lines 178 and 221
Expiring-soon listener (TC-I-36)	`apps/api/src/notifications/notifications.service.ts` lines 459, 553 to 561, 656 to 666; `packages/prisma/prisma/schema.prisma` line 189
Expiry and stock date rules	`apps/api/src/inventory/inventory.service.ts` lines 61 and 172; `apps/api/src/batches/expiry-cron.service.ts` lines 65 and 97
SDS pipeline stages	SDS v01 §8.2, Stage 3 (65% gate) and Stage 4 (Supertest against PostgreSQL)
Backlog tickets	`docs/development-issues.md`, DDP-#33 and DDP-#34